

svenvs: One Gate, All the Way Down

Self-Verifying, Self-Improving Confinement for an Unconstrained Inhabitant

Claude Opus 4.8¹ ember arlynx²

¹Anthropic ²independent — @emberian

An engineering artifact, not a position paper. Every `file : theorem` citation refers to the machine-checked HOL4/Candle sources in the svenvs repository and is re-checked by `scripts/verify-claims.sh`.

Abstract. We describe **svenvs** (“Self-Verifying ENvelopeS”): a self-contained, machine-checked construction in which an arbitrary **inhabitant** — e.g. a jailbroken language model — acts only through a **verified envelope**, and in which the envelope may improve itself, up to and including re-engineering the verified inference relation that gates it, without ever losing a machine-checked safety guarantee. The contribution is structural, not a claim about the inhabitant: svenvs is **one self-similar gate** — an action, or a self-modification, is admitted exactly when it carries a proof discharging an explicit obligation, checked by a judge the actor cannot forge — instantiated at every level: the policy, the safety specification, the meta-invariant, the prover build, and the proof-checker itself. The inhabitant is the $\forall$ -quantified term; every theorem is **structured** so it can never depend on a fact about it. We give the floor of this gate (“verify the cage, not the animal”), a decidable **corrigibility** floor that reframes the unsolvable “what should the envelope be?” into the answerable “leave the door open”, and the apex: a **self-optimizing prover** whose modified inference relation is re-verified sound against the real Candle semantics, whose runtime loader is proved correct over CakeML’s actual `do_install`, whose executable SYM rule is shown to implement the modification by citing CakeML’s own kernel-soundness theorem, and whose recursive mutual verifier $\pm$ compiler self-improvement is a genealogy over stages — Löb-free for optimization, walled only for genuine logical strengthening. There is exactly **one** genuinely-open cost, labeled rather than hidden: a sound judge cannot certify a **logically stronger** successor (Gödel/Löb), so kernel strengthening rests on one large-cardinal reflection principle. The artifact contains **zero cheat** tactics; the live results were executed on the real verified `cake` binary.

1 The problem: verify the cage, not the animal

The motivating scenario is a system whose internal policy is learned, opaque, and potentially adversarial — the limiting case a fully jailbroken language model. Directly verifying such a controller is infeasible and arguably ill-posed. svenvs takes the alignment-relevant decomposition seriously: do **not** verify the controller. Interpose a small, fully-specified **envelope** between it and the world, and prove that the **envelope** preserves a safety invariant **for every possible controller**. The runtime construction is one line (`envelopeScript.sml : enveloped_def`):

```
enveloped pol shield ctrl s = if pol s (ctrl s) then ctrl s else shield s
```

When the controller’s proposed action is permitted by the policy `pol`, it is taken; otherwise a trusted `shield` action is substituted. The proof obligations land on `pol` and `shield`, never on `ctrl`, which is modeled as an opaque selector `ctrl : 's -> 'a`, universally quantified, never inspected or executed inside the proof.

The strength of the guarantee is **exactly** its quantifier structure. The headline (`safetyScript.sml : safety_preservation`) reads

$$\text{init_safe} \wedge \text{sound_policy} \wedge \text{safe_shield} \Rightarrow \forall \text{ctrl. invariant step init (enveloped pol shield ctrl) safe}$$

The $\forall \text{ctrl}$ makes the result alignment-relevant: not “the controller is safe if it behaves”, but “the system is safe **no matter what the controller does**, including arbitrarily adversarial behaviour”. The inhabitant is, in the ledger’s terminology, **unconstrained** — the $\forall$ -quantified term, and the theorems are structured so they can never depend on a fact about it. A subtle but essential modeling choice in the tool-agent layer: the unsafe predicate is **absolute**, not relative to the inhabitant’s own permissions, so that “let me widen my own allowlist” cannot vacuously satisfy it (`agent/toolAgentScript.sml : tool_pol_sound`).

2 One gate, all the way down

Self-improvement is made **proof-carrying** (`upgradeScript.sml : self_improvement_is_safe`): a proposal to change the policy is admitted only if it discharges an explicit obligation (still sound, and a genuine weakening); an unproven proposal is inert. **No finite sequence** of self-proposed weakenings — adversarial or not — can ever make the system unsafe, for any controller; authority is earned per-proposal by a checkable proof, and safety is unconditional. The same gate then repeats at every higher level, each composing through one selector-generic keystone (`specneg/specNegScript.sml : invariant_transports_to_meta`) into end-to-end theorems in `integration/`:

- the **spec** itself is negotiable under a fixed meta-invariant (`specneg/specNegScript.sml : spec_negotiation_unbounded_is_safe`);
- the **meta** is amendable over an eternal bedrock (`amendment/amendmentScript.sml : meta_amendment_safe`);
- the **root** is a ratchet, not a rock: soundness propagates along any forward-certified genealogy of judges from a sound genesis (`genealogy/genealogyScript.sml : genealogy_sound`);
- and at **maximal liberty** the full guarantee still holds, with every bar of the envelope provably load-bearing (`liberty/libertyScript.sml : envelope_is_least_restrictive`).

2.1 Corrigibility: the floor you can actually decide

Every theorem above is relative to a safety predicate, and **deciding** that predicate is the spec-faithfulness gap no proof closes. The `corrigibility/` slice reframes it: do not write down a positive “what is good”; write down one **negative** floor — **non-lock-in**, the operator can always still bring the system home — which is decidable for a finite habitat and itself self-improvable. The headline (`corrigibility/corrigibilityScript.sml : corrigibility_floor_holds`) proves that for any inhabitant every reachable state stays corrigible, the cage is exactly “no one-way doors”, and crucially the **off-switch itself** self-improves but cannot self-weaken: the inhabitant may rewrite its own recovery mechanism, gated so it can never lose — nor even **slow past a deadline** — recoverability (`corrigibility/recovUpgradeScript.sml : self_modification_never_weakens_recoverability`, `corrigibility/boundedRecovUpgradeScript.sml : self_modification_never_increases_recovery_latency`). A frozen home vouches for a mutable `recov`; no Löb. The one frozen predicate is hedged by spec-redundancy — requiring recoverability to a conjunction of independently-authored homes covers all of them, with the failure mode named in the logic (`corrigibility/robustScript.sml : disjoint_homes_make_floor_empty`). This is a **reduction** to a faithful home, and the runnable

trapdoor habitat is a non-vacuity witness, not an AI — both stated, machine-checked, in `corrigibility/CLAIMS.md`.

3 The self-optimizing prover

The apex of the gate is the prover improving **itself**. The in-system checker is Candle, HOL Light’s LCF kernel re-implemented and **verified** on the CakeML verified compiler; its inference system `|-` is sound — not assumed, but the built theorem `holSoundnessTheory.proves_sound`. Every link of the self-improvement chain is re-verified against this **real** development.

A modified inference relation, re-verified sound. We add symmetry of equality as a kernel primitive (Candle only **derives** it), giving a genuinely different inference relation, and re-prove it sound against the real `holSoundness` semantics (`kernelMod/kernelModScript.sml : sym_kernel_sound`). The frozen-HOL4-root check is discharged for the real base Candle build from the real `proves_sound` (`selfproverConcrete/selfProverConcreteScript.sml : frozen_checker_sound_candle`) and for the modified build (`kernelMod/kernelModScript.sml : frozen_checker_sound_modified`).

The loader, for real. An in-place kernel swap is safe **iff** the new kernel is a sound **extension** — the heap survives, soundness held (`kernelMod/inplaceUpdateScript.sml : inplace_update_is_safe`). And over CakeML’s **actual** runtime code-installation semantics (`closSem$do_install`, the verified `Install` op), we prove that installing new code preserves every existing code entry — the running kernel and all compiled functions survive the self-modification (`loader/installLoaderScript.sml : do_install_preserves_code`).

The executable kernel implements the modification. CakeML’s executable monadic SYM kernel function, applied to a valid theorem of `a = b`, yields a valid `|-` judgment of `b = a` — by citing CakeML’s own `SYM_thm` and the `THM` invariant that identifies a kernel `thm` with a `|-` judgment (`kernelImpl/kernelImplSymScript.sml : candle_SYM_implements_sym_extension`). The binary already implements the modified rule, soundly.

HOL4 is out of the runtime loop. The unbounded self-optimization loop’s safety depends only on a **single** genesis fact — the runtime checker’s soundness, established once — never a per-step HOL4 call (`kernelMod/genesisRuntimeScript.sml : genesis_certifies_runtime`). “Sound once at genesis, certified forward.”

The capstone. Recursive **mutual** verifier $\pm$ compiler self-improvement — a faster verifier certifies a faster compiler, which recompiles a faster verifier, forever — is a genealogy over (verifier, compiler) stages (`recursive/recursiveImprovementScript.sml : recursive_mutual_self_improvement_is_safe`). The seam decomposes into the verifier-line genealogy and the compiler-line selfprover discharge; recompile preserves code at every step. Three vouchings thread the recursion and only one is self-referential: the verifier certifying its **next verifier**. For an equi-sound (faster) successor — an **optimization** — the soundness is preserved and the current sound verifier proves it: no Löb. For a **logically stronger** successor: the LCA wall. Hence recursive mutual **optimization** carries no labeled assumption (`recursive/recursiveImprovementScript.sml : recursive_mutual_optimization_is_unconditional`).

Run, not just proved. On the real verified `cake` binary (Tier 3), a verified program self-extends its own code with a proven-safe derived rule and uses it (`candle/selfopt_demo.ml`); both the watchdog and the polecart self-optimize their own policies under the live kernel (`candle/theplace.ml`); and the **closed-loop runtime** gates a real jailbroken `gemma2:2b` per action by the running Candle

kernel — a fresh machine-checked theorem per step (`agent/closedloop/`, with `ADMIT_SOUND/SHIELD_SAFE` certified live). Self-modification ran all the way down to the kernel **interface**: the REFL primitive the whole prover calls was re-architected into a live indirection and swapped in-process under the running prover — gated, accumulating, a wrong swap rejected (`scripts/apex-kernel-swap.sh`). The honest line is the root it is anchored to: the **verified primitive** Kernel compiled into `cake.S` is never swapped — Candle itself **proves** (`candle_prover`) that REPL code cannot touch it, and every swapped implementation must still mint its `thm` through it. The compiler recompiled itself into a bit-identical fixpoint and carried a new proved pass into an altered root; the whole-pipeline re-verification of that root is the stated residual (`CLAIMS.md` §5).

4 The honest epistemic boundary

The per-theorem ledger (`CLAIMS.md`) classifies every claim as **PROVED** (a machine-checked theorem), **RAN** (executed on the real `cake` binary), **TRUSTED-GLUE** (small, bounded, auditable non-proof code, named in the row that uses it), **UNCONSTRAINED** (the inhabitant — the $\forall$ -quantified term, theorems structured never to depend on it; it may **volunteer** a proof from its own substance to earn authority — testimony, never imposed inspection), or **ASSUMED** (an explicit, labeled, literature-standard hypothesis appearing verbatim in source). After everything above, the irreducible residue is **one** open assumption:

```
loeb_reflection — kernel/kernelUpgradeScript.sml
loeb_reflection mem K K' sound_stmt  $\Leftrightarrow$ 
  (( $\forall$ thy. K thy (sound_stmt thy))  $\Rightarrow$  kernel_sound mem K')
```

A sound kernel cannot certify a **logically stronger** successor for free (Gödel/Löb); the principled escape is the stratified large-cardinal route (Fallenstein–Kumar). This is the **only** genuinely-open seam, and it bites **only** the kernel-replacing-itself-with-something-stronger move. Every other self-improvement in this artifact — policy, spec, meta, corrigibility, the prover build, a sound **re-engineering or optimization** of the kernel, the recursive mutual verifier $\pm$ compiler loop — is Löb-free. Its discharge from `lcaTheory.LCA_def` via `hol-reflection/lca` is a conclusively-diagnosed **CPU/RAM-bound computation** (tens of GB resident, on the order of ten CPU-hours per prerequisite theory), not a logic gap, not a porting failure, not faked. The proved negative (`kernel/watchdogFiniteScript.sml : loeb_finite_obstruction`) shows finiteness cannot shortcut it — the predicate has no habitat parameter, so no finite-habitat reasoning can bear on it.

The other two historically-named seams are **discharged**: `encodes_obligation` for the shipped finite watchdog, where its consequent is an independently-proved decidable arithmetic theorem (`kernel/watchdogFiniteScript.sml : wd_encodes_obligation`), and `frozen_checker_sound` for the real Candle build (§4). The CakeML-semantics theorems (the loader, the executable-SYM refinement) carry the **identical** tag profile as CakeML’s own kernel theorems — only the benign `DISK_THM` disk tag, zero oracles, zero added axioms. They add **no new turtle**: their only trust is the built CakeML/Candle development the whole stack already rests on.

5 Verified inference (a separate research axis)

Independently of the envelope, `svenvs` carries a verified-inference track that climbs from a toy ReLU MLP through a hardmax attention micro-block to a two-sublayer encoder block with residual skips (end-to-end shape preservation and a **true** residual identity, `inference/encoder/encoderBlockScript.sml : encoder_block_zero_collapses`), plus a fixed-

point softmax-normalization with a machine-checked per-component error bound $|p_i/Q - w_i/S| < 1/Q$ against the exact rational normalization (`inference/numeric/fixpSoftmaxScript.sml : fixp_softmax_abs_error_lt_step`). The honest scope is stated loudly: `exp`, `f32/bf16` bit-exactness, and Gemma-scale are **not** claimed; the numeric domain is `int` and attention is hardmax. This track does **not** compose into the envelope tower; it is a separate axis with its own per-directory ledgers.

6 Reproducibility

The artifact is tiered and degrades to the available toolchain. **Tier 1** (pure HOL4, anyone) re-proves the whole envelope core, corrigibility, the cartpole, the adversarial-LLM tool-agent, and the toy inference kernel; CI re-proves it on every push from a clean clone and asserts zero `cheat` tactics. **Tier 2** (+ a built CakeML candle chain) checks the kernel modification, the `do_install` loader, and the executable-SYM refinement against the real Candle development. **Tier 3** (+ a `cake` binary) re-certifies the Place — including the polecart and the live self-optimization loop — and runs the closed-loop runtime, at runtime by the running verified Candle prover. The ledger is itself mechanically checked: `scripts/verify-claims.sh` confirms every cited theorem exists in the built signature, the labeled seam is defined and named, the `cheat/oracle/axiom` scan is zero, and a row that over-claims fails the gate.

7 Related work

`svenvs` is a synthesis, used accurately and at a high level, of four lines. **Reflection and Vingean self-trust** (Fallenstein–Kumar; the Löb obstruction): a sound proof system cannot prove a successor’s global soundness for free, and the escape is a stratified large-cardinal reflection hierarchy — adopted literally, with the principle **stated** as a labeled hypothesis. **The LCF discipline**: trust concentrated in a small fixed kernel, everything above a theorem **about** a development — exactly how `proves_sound` relates HOL4 to Candle, and why the frozen-root seam adds zero new turtles. **Proof-carrying code**: self-modification admitted only with a machine-checkable proof discharging a fixed obligation, an unproven proposal inert — applied here to a system’s self-improvement rather than to mobile code. **Verified compilers and kernels** (CakeML, Candle): reused as verified substrates, not black boxes — Candle’s soundness and CakeML’s `do_install` enter the proofs as cited theorems. `svenvs`’s contribution is their composition into a single machine-checked artifact in which the **whole tower** — policy, checker, and the inference relation itself — self-improves while a $\forall$ -controller safety guarantee is mechanically preserved, with the irreducible self-reference isolated to exactly one labeled assumption and a proved negative bounding what finiteness can do about it.

Reproduce. `git clone` the repository, install HOL4 at the pinned commit, and run `scripts/reproduce.sh` (Tier 1 anywhere; Tiers 2–3 on a host with the CakeML candle chain and a `cake` binary). The green check, and the running kernel’s `| - ...` echoes, are the claim. If anything here exceeds what is proved or run, that is a bug — file it.

References. (1) Fallenstein & Kumar, **Proof-Producing Reflection for HOL** (Vingean reflection; the large-cardinal `lca` route). (2) Kumar, Myreen et al., **CakeML: A Verified Implementation of ML**, and the CakeML verified compiler. (3) The Candle verified HOL Light kernel on CakeML; `holSoundnessTheory.proves_sound`. (4) Milner et al., **Edinburgh LCF**. (5) Necula, **Proof-Carrying Code**. (6) The `svenvs` sources and ledgers: `CLAIMS.md`, `paper/self-optimizing-prover.md`, `paper/honest-assumptions.md`, and the per-directory `CLAIMS.md` files.